

Context Pruning for Coding Agents via Multi-Rubric Latent Reasoning

Jingjing Wang*
Clemson University
jingjiw@clemson.edu

Xiwen Chen*
Morgan Stanley
xiwen.chen@morganstanley.com

Wenhui Zhu*
Arizona State University
wzhu59@asu.edu

Huayu Li
University of Arizona
hl459@arizona.edu

Zhengxiao He
University of Arizona
zhengxiaoh@arizona.edu

Feiyang Cai
Clemson University
feiyang@clemson.edu

Ana S. Carreon-Rascon
University of Arizona
anascarreonr@arizona.edu

Xuanzhao Dong
Arizona State University
xdong64@asu.edu

Feng Luo
Clemson University
luofeng@clemson.edu

Abstract

LLM-powered coding agents spend the majority of their token budget reading repository files, yet much of the retrieved code is irrelevant to the task at hand. Existing learned pruners compress this context with a single-objective sequence labeler, collapsing all facets of code relevance into one score and one transition matrix. We show that this formulation creates a modeling bottleneck: a single CRF transition prior must serve heterogeneous retention patterns, including contiguous semantic spans and sparse structural support lines. We propose **LaMR (Latent Multi-Rubric)**, a structured pruning framework that decomposes code relevance into two interpretable quality dimensions, semantic evidence and dependency support, each modeled by a dedicated CRF with dimension-specific transition dynamics. A mixture-of-experts gating network dynamically weights the per-rubric emissions conditioned on the query, and a final CRF layer on the fused emissions produces the aggregate keep-or-prune decision. To supervise each dimension without additional annotation cost, we derive multi-rubric labels from the existing training corpus via AST-based program analysis, simultaneously denoising the teacher’s binary labels. By effectively filtering distracting noise, LaMR frequently matches or even outperforms unpruned full-context baselines. Experiments on four benchmarks (SWE-Bench Verified, SWE-QA, LCC, LongCodeQA) show that LaMR wins 12 of 16 head-to-head multi-turn comparisons. It saves up to 31% more tokens on multi-turn agent tasks and improves Exact Match by up to +3.5 on single-turn tasks, while performance is frequently enhanced by denoising the context, and any remaining drops are marginal.

1 Introduction

LLM-powered coding agents are rapidly reshaping software engineering, with state-of-the-art (SOTA) frameworks such as SWE-agent [1], OpenHands [2], and Agentless [3] automating complex, repository-level tasks. Commercial success followed suit: Anthropic’s Claude Code and Cursor have reached multi-billion dollar valuations by processing millions of lines of code daily. However,

*Equal contribution.

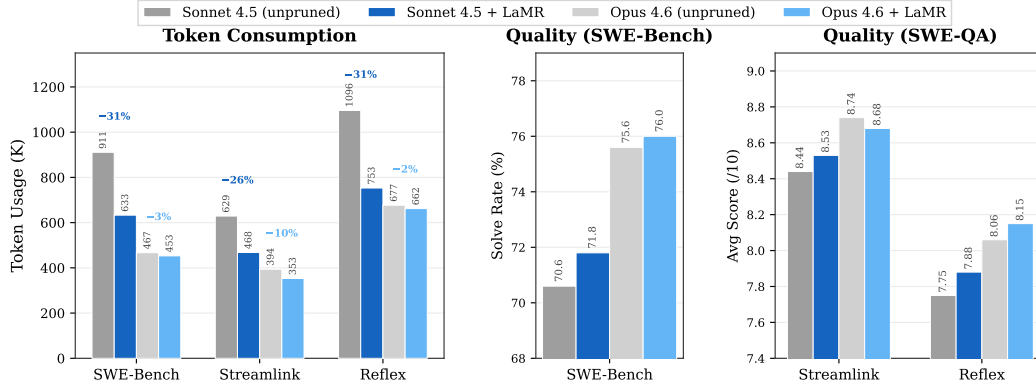


Figure 1: Effect of LaMR across two backbone models and three benchmarks. **Left:** Token consumption (K tokens); percentage labels show reduction by LaMR relative to unpruned. **Middle/Right:** Task quality (SWE-Bench solve rate in %; SWE-QA average score on a 10-point scale). Gray bars are unpruned baselines; blue bars are with LaMR. LaMR consistently reduces token usage while maintaining or slightly improving quality.

these agents have hit a *context wall*: they spend approximately 67–76% of their token budget simply reading files [4]. This context bloat not only incurs prohibitive API costs but also triggers "attention dilution" and the *lost-in-the-middle* phenomenon [5], where irrelevant code segments degrade the agent’s reasoning precision.

The quest for context efficiency has evolved through three distinct paradigms. Initially, task-agnostic heuristics were employed, such as perplexity-based token pruning [6, 7] or similarity-based RAG [8]. However, as noted by [9], these methods primarily target natural language and face critical limitations when applied to code: token-level pruning frequently compromises syntactic validity, while abstractive techniques discard character-level details essential for debugging. Furthermore, these approaches operate with static compression ratios and task-agnostic criteria, failing to adapt to the evolving informational needs of multi-turn agent interactions [1].

Recently, SWE-Pruner [4] mitigated this issue by framing code context pruning as a structured sequence labeling task. It trains a 0.6B-parameter neural skimmer by minimizing the conditional random field negative log likelihood (CRF-NLL) [10]. This allows adaptive token usage reduction while keeping character-level integrity. However, this single-objective formulation introduces an inherent conflict: it uses one transition matrix to model all types of code relevance. Code relevance operates across distinct dimensions. Semantic relevance usually forms continuous blocks, which require high self-transition probabilities in the CRF. In contrast, structural dependencies (like `import` statements) and control-flow pairs (like `try/except`) are scattered. A single transition matrix creates a modeling bottleneck: it must serve both dense semantic spans (contiguous keep-blocks) and sparse structural support (scattered single-line jumps) with one set of parameters. Forced to compromise, the model biases toward the denser semantic blocks. It retains the main function body but discards necessary structural lines (fig. 2). This missing structure severely impacts strong coding agents. When models like Claude 4.6 encounter broken syntax or missing context, they fail and repeatedly retry the task. These iterative retries end up consuming more tokens than the unpruned baseline.

To address this, we propose LaMR (**Latent Multi-Rubric**). We decompose code relevance into K latent rubrics, each modeled by a dedicated CRF head with its own transition matrix. This allows dimension-specific sequence patterns. A query-adaptive Mixture-of-Experts (MoE) gate then combines the per-dimension CRF emissions, dynamically balancing semantic and structural signals for the current task. Since existing datasets provide only binary keep-or-prune masks, we introduce Rubric-Guided Labeling to derive fine-grained supervision without additional annotation. Using Abstract Syntax Tree (AST) analysis, we extract dimension-specific labels from the existing masks and recover structurally necessary lines that the teacher may omit, such as imports, class headers, and control-flow companions. This produces a cleaner training signal for the neural skimmer. More broadly, LaMR is not merely deleting tokens; it aims to preserve self-contained evidence-support units: task-relevant lines together with the structural context needed to interpret them. By effectively

extracting signal from noise, LaMR provides agents with higher-quality observations. We empirically demonstrate that this structure-aware approach not only compresses the context but carries the potential to elevate the agent’s final task performance.

We summarize our contributions below: (i) **Latent Multi-Rubric Formulation**. We identify the transition conflict in single-objective pruning and resolve it by modeling code relevance through K separate CRF heads. (ii) **Query-Adaptive Gating**. We use an MoE gating network to adjust the focus between semantic and structural code based on the agent’s query. (iii) **AST-based Label Denoising**. We develop a zero-cost pipeline to extract multi-dimensional labels from binary masks, which corrects missing structural tokens in the teacher data. (iv) **Superior Efficiency-Performance Tradeoff**. We extensively evaluate LaMR across four benchmarks and two state-of-the-art LLM backbones. As a lightweight middleware, it consistently mitigates token bloat and achieves a strictly better tradeoff between context compression and downstream task quality compared to prior methods.

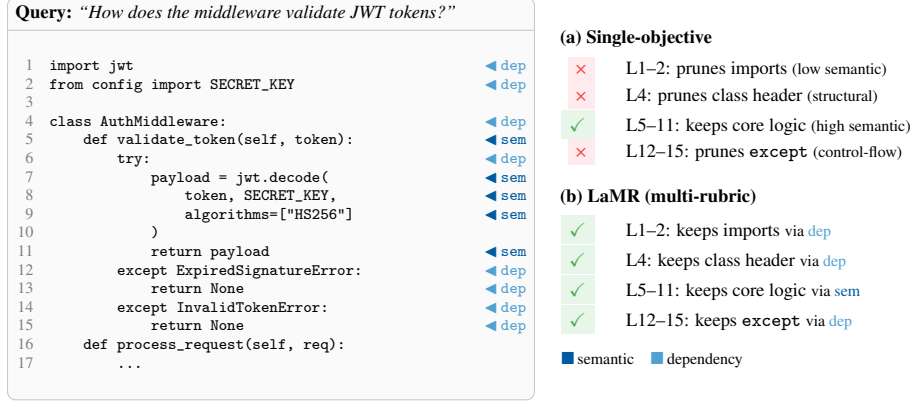


Figure 2: A single-objective pruner collapses semantic and structural relevance into one score, discarding lines that rank low on semantics alone. LaMR decomposes relevance into two rubrics (semantic and dependency), each with a dedicated CRF, and fuses them via query-adaptive gating.

2 Preliminaries

Code context pruning. Given a code context $C = (x_1, \dots, x_n)$ of n tokens and a natural-language query q describing the agent’s current information need, the task is to produce a binary label sequence $\mathbf{y} \in \{\text{keep}, \text{prune}\}^n$ such that the retained tokens preserve query-relevant information while minimizing context length. Token-level scores are aggregated to line granularity and thresholded to produce the final pruned output.

From heuristic to learned pruning. Prior approaches rely on task-agnostic heuristics: LLM-Lingua [6] and Selective-Context [7] prune tokens based on perplexity or self-information, while RAG-based methods retrieve code chunks via embedding similarity [11]. These methods use surface-level statistics without understanding code structure or the agent’s intent. SWE-Pruner [4] takes a fundamentally different approach: it trains a lightweight neural skimmer that *learns* query-conditioned pruning as structured sequence labeling with a linear-chain CRF [10]. A CRF models the conditional probability of the full label sequence jointly, rather than predicting each token independently:

$$P(\mathbf{y} | \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp\left(\sum_{t=1}^n \phi(y_t, \mathbf{x}, t) + \sum_{t=2}^n \psi(y_{t-1}, y_t)\right), \quad (1)$$

where $\phi(y_t, \mathbf{x}, t)$ are *emission potentials* that score how likely token t is to be kept or pruned based on its content alone, and $\psi(y_{t-1}, y_t)$ are *transition potentials* stored in a learnable 2×2 matrix that bias adjacent tokens toward label consistency (e.g., preferring consecutive keeps over isolated ones). $Z(\mathbf{x})$ is a normalizing constant computed via the forward algorithm. This structured formulation avoids fragmented pruning and achieves 23–38% token reduction on SWE-Bench Verified while improving task success rates.

Limitations of single-objective pruning. Despite its effectiveness, the single-objective CRF formulation has an inherent limitation: it relies on one set of emission scores and one transition

matrix, collapsing multiple aspects of code relevance into a single scalar. As illustrated in Figure 2, when a query targets token validation logic, the model correctly identifies the core function body as relevant but discards `import` statements (low semantic score, but structurally necessary), class headers (structural scaffolding), and `except` clauses (low semantic relevance, but contextually paired with the `try` block). These failures stem from a fundamental tension: a single transition matrix must simultaneously model (i) contiguous semantic blocks and (ii) non-contiguous dependency chains (including paired control-flow structures). These failures reflect a modeling bottleneck: one transition prior must serve both contiguous semantic spans and sparse dependency-support lines. Because the model does not explicitly distinguish why a line should be retained, it can favor dense semantic blocks while under-preserving structural support needed to interpret them.

3 Method

Key Insight

Code relevance is not a single quantity. A line may be retained for its *semantic* match to the query, or for its role in a *structural dependency* chain (imports, scope headers, paired control flow). These aspects exhibit distinct sequential dynamics that a single CRF transition matrix cannot accommodate. LaMR assigns each aspect its own CRF head with an independent transition matrix, and fuses them via a query-adaptive gating network.

We decompose code relevance into K quality dimensions $\mathcal{O} = \{o_1, \dots, o_K\}$, each capturing a distinct reason to retain a line (Figure 2). In practice we use $\mathcal{O} = \{\text{semantic}, \text{dependency}\}$ ($K=2$): *semantic* captures topical relevance to the query, while *dependency* captures structural necessity (imports, definitions, scope headers, and control-flow siblings such as `try/except`). We find that these two dimensions already subsume the contextual-coherence signals illustrated in Figure 2, since AST-based dependency tracing naturally propagates through enclosing scopes and paired control-flow branches (§3.1). We call these dimensions *latent rubrics*: they are induced during training via supervision derived from the teacher LLM’s retention decisions, conditioned on static program analysis (§3.1), but at inference the model produces per-rubric emissions entirely from learned parameters, without requiring program analysis for the scoring itself (AST is only used in a lightweight post-processing step; see §3.4). The model first extracts a shared token representation via multi-layer feature fusion (§3), then projects it into per-dimension emissions fed to K independent CRF heads (§3), whose outputs are combined by an MoE gating network (§3) and decoded through a final fused CRF (§3). We describe each component below.

Multi-layer feature fusion. Following SWE-Pruner, we adopt the same backbone encoder (a 0.6B-parameter reranker based on Qwen3-Reranker-0.6B [12] with L transformer layers). The query q and code context C are concatenated into a single input sequence, so all hidden representations are inherently query-conditioned. Let $\mathbf{H}^{(l)} \in \mathbb{R}^{n \times d}$ denote the hidden-state matrix at the l -th layer. To provide richer per-token representations, we extract hidden states from three layers (an early layer l_e , a middle layer l_m , and the final layer L), concatenate them, and refine the result through N_{fuse} multi-head self-attention blocks with residual connections:

$$\mathbf{H}^{\text{cat}} = [\mathbf{H}^{(l_e)} \parallel \mathbf{H}^{(l_m)} \parallel \mathbf{H}^{(L)}] \in \mathbb{R}^{n \times 3d}, \quad (2)$$

$$\mathbf{H}^{(i+1)} = \text{LN}(\mathbf{H}^{(i)} + \text{MHA}(\mathbf{H}^{(i)})), \quad i = 0, \dots, N_{\text{fuse}} - 1, \quad (3)$$

where $\parallel$ denotes concatenation and $\mathbf{H}^{(0)} = \mathbf{H}^{\text{cat}}$. In practice, $l_e = \lfloor 0.25L \rfloor$ and $l_m = \lfloor 0.5L \rfloor$ (layers 7 and 14 for the 28-layer backbone). The resulting per-token representation $\mathbf{h}_t \in \mathbb{R}^{3d}$ serves as input to both the emission heads and the gating network.

Per-objective CRF heads. A shared MLP maps the fused representations to per-objective emission vectors $\mathbf{E} = \text{MLP}(\mathbf{h}) \in \mathbb{R}^{n \times K \times 2}$, where $\mathbf{E}_{t,k} \in \mathbb{R}^2$ scores how likely token t should be kept or pruned from the perspective of rubric dimension k . Crucially, each dimension k maintains its own learnable CRF parameters: a transition matrix $\mathbf{T}^{(k)} \in \mathbb{R}^{2 \times 2}$ encoding which label transitions (keep→keep, keep→prune, etc.) are natural for that dimension, along with start potentials $\pi^{(k)} \in \mathbb{R}^2$ and end potentials $\omega^{(k)} \in \mathbb{R}^2$ that bias the first and last token’s label respectively. The path score under dimension k is:

$$S_k(\mathbf{x}, \mathbf{y}) = \pi_{y_1}^{(k)} + \sum_{t=1}^n \mathbf{E}_{t,k,y_t} + \sum_{t=2}^n \mathbf{T}_{y_t, y_{t-1}}^{(k)} + \omega_{y_n}^{(k)}. \quad (4)$$

Intuitively, the emission term asks “is this token relevant under dimension k ?” while the transition term asks “given the previous decision, should the label change here?” By design, different dimensions can learn qualitatively different transition behaviors. We expect *semantic* to favor long runs of the same label (topically related code is contiguous), while *dependency* tolerates frequent alternation (imports, scope headers, and their usage sites are scattered; paired structures like `try/except` are also captured through dependency tracing). A single transition matrix cannot accommodate both regimes simultaneously, which is precisely the limitation we address.

Mixture-of-experts gating. Queries with different intents require different emphasis across the K objectives. A lightweight gating network produces per-token weights $\mathbf{g}_t = \text{Softmax}(\text{MLP}_{\text{gate}}(\mathbf{h}_t)) \in \Delta^{K-1}$, and the fused emission at each position is:

$$\mathbf{e}_t^{\text{fused}} = \sum_{k=1}^K g_{t,k} \mathbf{E}_{t,k} \in \mathbb{R}^2. \quad (5)$$

To prevent gate collapse onto a single dominant objective, we regularize with an entropy penalty $\mathcal{L}_{\text{gate}} = \log K - \mathbb{H}[\bar{\mathbf{g}}]$, where $\bar{\mathbf{g}} = |\mathcal{V}|^{-1} \sum_{t \in \mathcal{V}} \mathbf{g}_t$ is the mean gate vector over valid positions and $\mathbb{H}[\cdot]$ denotes Shannon entropy.

Fused CRF. The per-objective CRFs model dimension-specific transition dynamics during training, but the final pruning decision must be a single coherent sequence. When $K > 1$, the gated emissions $\{\mathbf{e}_t^{\text{fused}}\}_{t=1}^n$ are therefore passed through a *final CRF layer* with its own transition matrix $\mathbf{T}^f \in \mathbb{R}^{2 \times 2}$ and start/end potentials. This layer captures cross-dimension sequential dependencies that may not be expressible within any single rubric’s transition matrix alone. The corresponding loss is:

$$\mathcal{L}_{\text{main}} = \frac{1}{n} (\log Z_f(\mathbf{x}) - S_f(\mathbf{x}, \mathbf{y})), \quad (6)$$

where $\mathbf{y}$ is the aggregate binary label sequence.

Document-level scoring. As an auxiliary task that encourages the backbone to maintain a global view of document relevance alongside fine-grained token pruning, we preserve the backbone reranker’s document-level relevance estimation. The last token’s hidden state is projected onto the vocabulary embeddings to extract the log-probability of “yes” relative to “no”: $s_{\text{doc}} = \log \sigma(\mathbf{h}_n^\top \mathbf{w}_{\text{yes}} - \mathbf{h}_n^\top \mathbf{w}_{\text{no}})$. This score is supervised with $\mathcal{L}_{\text{score}} = \text{MSE}(e^{s_{\text{doc}}}, s_{\text{ref}})$, where $s_{\text{ref}} \in [0, 1]$ is a teacher-provided relevance score.

3.1 Training Data and Rubric-Guided Labeling

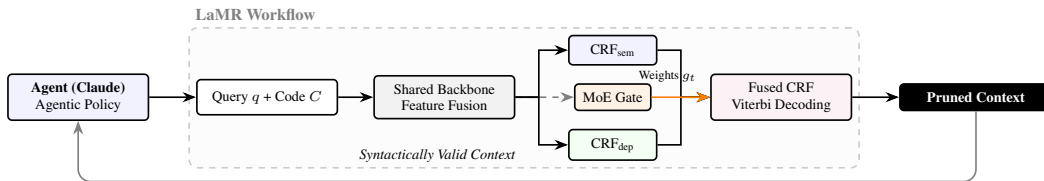


Figure 3: **The LaMR workflow.** Operating as an agentic middleware, it intercepts file reads, routes features through parallel latent rubrics, and returns a syntactically pruned context via the loop.

Base training corpus. We build on the training set constructed by SWE-Pruner [4]: code snippets sampled from high-quality GitHub repositories are paired with task-oriented queries synthesized by a teacher LLM. Each example contains a line-level retention mask $\mathbf{y}$ and a document-level relevance score s_{ref} . An LLM-as-a-Judge filter retains high-quality examples, yielding approximately 61K training quadruplets $(q, C, \mathbf{y}, s_{\text{ref}})$. These aggregate labels supervise the main pruning loss $\mathcal{L}_{\text{main}}$ and the relevance-score loss $\mathcal{L}_{\text{score}}$. However, the aggregate mask collapses different reasons for retention into a single binary decision: a line may be kept because it is direct query evidence, or because it structurally supports another retained line. Our deployed sem+dep model is trained on a rubric-enriched extension of this corpus. For each example, we construct per-line rubric labels $\{\mathbf{y}^{(o)}\}_{o \in \mathcal{O}}$ using the retained-line mask together with static program analysis. Although the rubric builder can produce multiple diagnostic dimensions, the deployed checkpoint uses the name-projected semantic and dependency labels to supervise $\mathcal{L}_{\text{rubric}}$.

Rubric decomposition. We extend this corpus with multi-dimensional *rubric scores* derived from static program analysis, requiring zero additional LLM calls. For each code file, we parse the AST and compute per-line scores along each rubric dimension: (i) *semantic*: lines in the teacher’s retained set receive the document-level relevance score; all other lines receive zero; (ii) *dependency*: starting from retained lines, we trace symbol definitions, imports, call sites, and enclosing scope headers via AST-extracted dependency edges, propagating scores through h hops with geometrically decaying weights. For retained lines inside compound statements (`if/try/with/for`), we additionally include the enclosing header and sibling branches (`else`, `except`, `finally`), so that control-flow coherence is captured within this dimension. The resulting rubric vector $\mathbf{r}_j \in [0, 1]^K$ for line j is binarized at threshold 0.5 to obtain per-objective CRF labels that supervise $\mathcal{L}_{\text{rubric}}$.

Rationale for label decomposition. Although all rubric dimensions derive from the same teacher-provided binary mask, the decomposition provides three concrete benefits. First, it corrects implicit label noise: lines the teacher marks as prunable on semantic grounds (e.g., `import` statements) are recovered by the dependency dimension via AST edges, ensuring structural integrity despite the teacher’s semantic focus, giving the model a cleaner supervisory signal for structurally necessary code. Second, it *decouples conflicting transition dynamics*: a single CRF transition matrix cannot simultaneously favor high self-transition (contiguous semantic blocks) and frequent alternation (scattered dependency chains and control-flow siblings); separate per-objective CRFs eliminate this optimization conflict. Third, it acts as a *structural regularizer*: by forcing the model to predict interpretable intermediate representations, the rubric loss reduces overfitting to spurious correlations in the aggregate label. Importantly, this rubric construction, conditioned on teacher labels and augmented by AST analysis, is used *only during training*; at inference, the model’s per-rubric CRF heads and gating network have internalized these quality dimensions as latent representations, producing multi-rubric scores without rubric-level program analysis.

3.2 Theoretical Motivation

We give two propositions that formalize why a single scalar relevance score is insufficient for code pruning and why dependency support should be treated differently from optional context. Proofs are in Appendix E.

Proposition 1 (Conditional utility of support lines). *Let e be an evidence line and d a support line required by e . Under the utility $U(S) = \mathbf{1}[e \in S] - M \mathbf{1}[e \in S, d \notin S] - \lambda|S|$ with missing-support penalty $M > \lambda > 0$, the marginal value of retaining d is positive when e is retained and negative when e is not: $\Delta(d \mid e \in S) = M - \lambda > 0$, $\Delta(d \mid e \notin S) = -\lambda < 0$. A single unconditional relevance score cannot represent this conditional behavior.*

Proposition 2 (Dependency-closed optimality). *Let $G = (V, E)$ be a dependency graph over code lines, where $(i, j) \in E$ means line i requires line j . Under the utility $U(S) = \sum_{i \in S} r_i - \lambda \sum_{i \in S} c_i - M \sum_{(i,j) \in E} \mathbf{1}[i \in S, j \notin S]$, if $M > \max_j (r_j - \lambda c_j)_+$, then every optimal S^* is dependency-closed: $i \in S^*, (i, j) \in E \Rightarrow j \in S^*$.*

Proposition 1 explains why LaMR separates semantic evidence from dependency support: `import` and scope lines may have low semantic relevance but high conditional value. Proposition 2 justifies AST-aware repair as an approximation to dependency closure over high-confidence structural edges.

3.3 Training Objective

The training loss has two core components:

$$\mathcal{L} = (1 - \lambda_r) \mathcal{L}_{\text{main}} + \lambda_r \mathcal{L}_{\text{rubric}} \quad (7)$$

$\mathcal{L}_{\text{main}}$ is the CRF negative log-likelihood on the gating-fused emissions (eq. (6)), supervising the aggregate keep-or-prune decision with the teacher’s binary labels. $\mathcal{L}_{\text{rubric}} = \sum_{k=1}^K w_k \mathcal{L}_{\text{CRF}}^{(k)} / \sum_{k=1}^K w_k$ is the weighted sum of per-rubric CRF-NLL terms, where each $\mathcal{L}_{\text{CRF}}^{(k)}$ operates on the k -th CRF head’s emissions with the corresponding rubric labels (weights $w_{\text{sem}} = 1.0$, $w_{\text{dep}} = 0.7$). The interpolation coefficient $\lambda_r = 0.6$ gives slightly more weight to rubric supervision, encouraging the model to learn dimension-specific transition patterns rather than relying solely on the aggregate signal. Two auxiliary losses (document-level relevance scoring and gate entropy regularization) are added with small coefficients; we detail these in Appendix D.5.

3.4 Inference

At inference time, LaMR acts as a lightweight middleware between the coding agent and its environment. When the code context exceeds the model’s maximum input length, it is split into overlapping chunks that are processed independently. Token decisions in overlapping regions are averaged, and the document-level relevance score is taken as the maximum across chunks. For each chunk, the backbone encoder and fusion layers produce the fused representation $\mathbf{h}_t$. The K per-objective CRF heads compute emissions, the gating network produces per-token objective weights, and the final CRF performs *Viterbi decoding* on the fused emissions (Eq. 5), yielding a binary keep/drop decision $\hat{y}_t \in \{0, 1\}$ for each token (see Appendix C for details). Since pruning operates at line granularity, token-level decisions are mapped back to source-code character offsets and averaged into line-level keep fractions. A line l_j is retained if its aggregated score is at least τ (we set 0.4). Finally, an AST-aware post-processing pass repairs the pruned code by adding back scope headers, control-flow headers, dependency-related definitions such as referenced imports, bracket/string closures, and selected return lines, and by replacing removed regions with placeholder comments or pass statements.

4 Experiments

4.1 Setup

We adopt the same evaluation protocol as SWE-Pruner [4] for direct comparison.

Benchmarks. We evaluate on four benchmarks spanning two regimes. For *multi-turn agent tasks*, we use SWE-Bench Verified [13], a set of 500 real GitHub issues requiring multi-file patch generation, and SWE-QA [14], which tests repository-level question answering across Streamlink, Reflex, and Conan. Compared with the original SWE-Pruner evaluation, which uses Claude Sonnet 4.5 and GLM-4.6, we evaluate with Claude Sonnet 4.5 and Claude Opus 4.6. The latter provides a stronger stress test: the unpruned Opus 4.6 agent already reaches 75.6% on SWE-Bench Verified, so pruning must improve efficiency without degrading a strong agent. For *single-turn tasks*, we use LCC [15], containing 500 Python completion examples with 5K+ token contexts, and LongCodeQA [16], which evaluates question answering over code contexts up to 1M tokens. Both are evaluated under $4\times$ and $8\times$ compression constraints.

Baselines. We compare against six methods: LLMingua-2 [17] and Selective-Context [7] (token-level pruning), RAG retrieval via UniXcoder embeddings [11], LongCodeZip [9] (hierarchical code compression), LLM Summarize (GPT-based context summarization), and SWE-Pruner [4] (single-objective CRF, our direct baseline). Full Context and No Context serve as upper and lower bounds. To ensure fair comparison under modern model and runtime behavior, we *reproduce* SWE-Pruner in the same evaluation stack as LaMR (same benchmark splits, agent harnesses, model endpoints, and runtime). Our primary conclusions are drawn from this reproduced baseline rather than published numbers, which may depend on model versions and API behavior that are difficult to replicate exactly.

Evaluation protocol. For single-turn tasks, we follow the prescribed $4\times$ and $8\times$ compression settings and report Edit Similarity (ES), Exact Match (EM), Accuracy, and compression ratio ($1/\tau$). For multi-turn tasks, we report task performance together with total token consumption, interaction rounds, and relative changes (Δ). All methods are evaluated under matched compression constraints.

Implementation details. We initialize from the same Qwen3-Reranker-0.6B backbone as SWE-Pruner. LaMR uses $K=2$ rubrics, semantic and dependency, with softmax gating and a final fused CRF. We train for 3 epochs on 8 GPUs with aggregate and rubric-specific CRF-NLL losses, $\lambda_s=0.05$, $\lambda_r=0.6$, $\lambda_g=0.002$. For multi-turn evaluation, LaMR is deployed as a lightweight middleware between the coding agent and its environment: we use Mini SWE Agent [1] for SWE-Bench Verified and OpenHands [2] for SWE-QA. For single-turn tasks (LCC and LongCodeQA), the pruner is served through an HTTP endpoint in the online reranking pipeline. Full hyperparameters are in Appendix D.

Table 1: Results on SWE-Bench Verified. LaMR achieves comparable success rates with substantially fewer tokens (large efficiency win, small performance gap).

Agent	Rounds	Solved	Success (%)		Tokens (M)	
			Value	Δ	Value	Δ
Mini SWE Agent (Sonnet 4.5)	51.0	353/500	70.6	–	0.911	–
+ SWE-Pruner	41.7	360 /500	72.0	$\uparrow 1.4$	0.701	$\downarrow 23.1\%$
+ LaMR	39.6	359/500	71.8	$\uparrow 1.2$	0.633	$\downarrow 30.5\%$
Mini SWE Agent (Opus 4.6)	30.9	378/500	75.6	–	0.467	–
+ SWE-Pruner	26.8	383 /500	76.6	$\uparrow 1.0$	0.498	$\uparrow 6.6\%$
+ LaMR	25.8	380/500	76.0	$\uparrow 0.4$	0.453	$\downarrow 3.0\%$

Table 2: Results on SWE-QA across three repositories. LaMR reduces rounds and tokens while maintaining or improving scores.

Method	Claude Sonnet 4.5			Claude Opus 4.6		
	Score (Δ)	Rounds	Tokens (K, Δ)	Score (Δ)	Rounds	Tokens (K, Δ)
<i>Streamlink</i>						
Unpruned	8.44	23.4	629.3	8.74	17.4	394.4
+ SWE-Pruner	8.59 $\uparrow 0.15$	23.9	587.1 $\downarrow 6.7\%$	8.60 $\downarrow 0.14$	19.0	398.6 $\uparrow 1.1\%$
+ LaMR	8.53 $\uparrow 0.09$	16.3	467.7 $\downarrow 25.7\%$	8.68 $\downarrow 0.08$	18.5	353.2 $\downarrow 10.4\%$
<i>Reflex</i>						
Unpruned	7.75	33.2	1096.3	8.06	26.9	677.0
+ SWE-Pruner	8.07 $\uparrow 0.32$	32.4	901.5 $\downarrow 17.8\%$	8.14 $\uparrow 0.08$	30.7	737.0 $\uparrow 8.9\%$
+ LaMR	7.88 $\uparrow 0.13$	29.2	753.2 $\downarrow 31.3\%$	8.15 $\uparrow 0.09$	27.4	661.9 $\downarrow 2.2\%$
<i>Conan</i>						
Unpruned	8.35	23.9	672.9	8.68	16.2	301.9
+ SWE-Pruner	8.37 $\uparrow 0.02$	23.5	551.7 $\downarrow 18.0\%$	8.61 $\downarrow 0.07$	18.6	368.0 $\uparrow 21.9\%$
+ LaMR	8.48 $\uparrow 0.13$	18.8	535.2 $\downarrow 20.5\%$	8.71 $\uparrow 0.03$	15.1	325.9 $\uparrow 7.9\%$

4.2 Main Results

Overall Result

Across 16 head-to-head multi-turn comparisons with SWE-Pruner (8 agent settings $\times$ 2 dimensions: quality and efficiency), **LaMR wins 12 and loses 4**. In single-turn tasks, the advantage is even clearer, with LaMR sweeping all 8 performance metrics. Crucially, by effectively filtering distracting noise, LaMR frequently matches the unpruned full-context baselines, and even outperforms them on benchmarks like LongCodeQA and SWE-Bench. The efficiency wins are large: LaMR saves **7% to 14% more tokens** on multi-turn tasks and improves **EM by up to +3.5** on single-turn tasks. The isolated losses are marginal (≤ 0.2 on a 10-point score scale, ≤ 0.6 pp on success rate). Moreover, SWE-Pruner has a much higher chance of *increasing* total token usage when paired with stronger backbone models (4 of 5 multi-turn Opus 4.6 settings, up to +22%), undermining its utility as a pruner.

SWE-Bench Verified. Table 1 compares agent performance with and without pruning. With Sonnet 4.5, SWE-Pruner edges ahead on success rate by 0.2 pp (72.0% vs. 71.8%), but LaMR saves substantially more tokens (30.5% vs. 23.1%, a 7.4pp gap) and reduces interaction rounds from 51.0 to 39.6 (22% fewer). The pattern is sharper with Opus 4.6: SWE-Pruner *increases* token usage by 6.6%, likely because aggressive pruning removes structurally necessary code and forces the agent into repeated file reads. LaMR avoids this failure mode, reducing tokens by 3.0% with only a 0.6pp lower success rate. In both settings the efficiency gap is large while the performance gap is negligible.

SWE-QA. Table 2 reports results across three repositories and two backbones. LaMR improves token efficiency in all six settings, saving 2–19pp more than SWE-Pruner on average. On score, LaMR wins 4 of 6 settings; when SWE-Pruner scores higher (Streamlink and Reflex with Sonnet), the margins are small (0.06 and 0.19 on a 10-point scale). With Opus 4.6, SWE-Pruner increases token usage in all three repositories (+1%, +9%, +22%), whereas LaMR reduces or limits overhead. This suggests that single-objective pruning can remove code needed by stronger models, causing extra file reads, while LaMR preserves more useful observations.

Single-turn tasks. Table 3 reports results on LCC and LongCodeQA. LaMR improves task quality in all single-turn comparisons: ES and EM on LCC under both $4\times$ and $8\times$, and accuracy on LongCodeQA under both constraints. The gains are substantial, including +3.5 EM and +1.63 ES on LCC at $8\times$, and +1.80 accuracy on LongCodeQA at $4\times$. SWE-Pruner sometimes compresses more aggressively (e.g., $10.92\times$ vs. $9.17\times$ on LCC $8\times$), but at a clear cost to task quality. Token-level baselines (LLMLingua-2, Selective-Context) degrade more sharply at high compression, while LaMR maintains stable performance through line-level granularity that preserves syntactic structure.

Table 3: Results on LCC and LongCodeQA. LaMR achieves the best task performance across all settings while maintaining high compression ratios.

Method	Long Code Completion						Long Code QA			
	4× Constraint			8× Constraint			4× Constraint		8× Constraint	
	1/τ	ES	EM	1/τ	ES	EM	1/τ	Acc	1/τ	Acc
Full	1.0	64.65	40.5	1.0	64.65	40.5	1.0	54.05	1.0	54.05
No Context	∞	44.90	13.5	∞	44.90	13.5	∞	38.39	∞	38.39
Selective-Context	3.27	52.48	22.0	7.49	48.67	17.0	3.69	55.36	7.32	51.79
LLMLingua-2	3.32	49.47	15.5	7.89	44.74	13.0	3.57	55.36	7.68	51.33
RAG	3.29	58.97	30.5	6.60	55.82	29.0	3.06	58.04	5.87	55.86
LongCodeZip	2.77	57.77	28.0	7.85	56.08	27.5	3.98	52.25	7.39	54.95
SWE-Pruner	5.56	58.63	31.5	10.92	57.58	31.0	13.95	59.46	14.84	58.71
LaMR	4.37	61.15	35.5	9.17	59.21	34.5	16.48	61.26	18.51	60.00

4.3 Ablation Study

We ablate three design choices in LaMR: the learned objective set, CRF-structured prediction, and AST-aware structural repair. The ablations evaluate LaMR as a hybrid neural-symbolic pruner rather than attributing all gains to a single module. The no-AST variant isolates learned semantic/dependency scoring without structural repair, while the full model combines learned role-aware scoring with AST-aware restoration. The gap between these variants shows that repair contributes under compression. At the same time, semantic-only and context-based variants remain below semantic+dependency, indicating that learned objective decomposition also matters. Table 4 summarizes the main trends; full precision/recall, compression, token, and constructor results are provided in Appendix F.

Objective selection. Table 4 shows that semantic relevance alone is insufficient: the semantic-only model is competitive but consistently below LaMR, indicating that direct relevance does not recover all support lines needed for useful compressed code. Context-based objectives are also competitive, but they do not outperform semantic+dependency. This supports the distinction between context and dependency: context is often local and optional, whereas dependency support can be non-local and structurally necessary. Adding context as a third objective does not improve the final tradeoff, suggesting that context is better handled through retrieval geometry, local windows, and rendering rather than as a universal learned objective.

Structured prediction and AST repair. The detailed validation ablation in Table 7 shows that the FFN variant is high precision but low recall, while CRF structure recovers recall and improves F1. Removing AST-aware repair reduces validation F1, lowers LongCodeQA accuracy under aggressive compression, and increases Conan interaction rounds. Thus, dependency-aware scoring and AST repair are complementary: the learned model scores evidence and support lines, while AST repair restores high-confidence structural dependencies after thresholding.

Table 4: Ablation summary. LCQA denotes Long-CodeQA; Rnd denotes Conan rounds. Full tables are in Appendix F.

Variant	Val F1 ↑	LCQA 4× Acc. ↑	LCQA 8× Acc. ↑	Conan Score/Rnd
Sem only	0.777	56.25	58.55	8.13 / 21.6
Sem + Ctx	0.788	59.26	56.48	8.26 / 21.3
Ctx + Dep	0.786	60.36	51.82	8.32 / 21.3
Sem+Dep+Ctx	0.784	57.27	53.21	8.31 / 21.6
Sem+Dep w/o AST	0.780	59.09	55.56	8.37 / 21.3
LaMR	0.796	61.26	60.00	8.48 / 18.8

Robustness across context constructors. We also test LaMR with different upstream constructors; full results are in Table 10. LaMR remains effective with both RAG-style retrieval and LongCodeZip-style structural retrieval. The strongest LCC result is obtained when LongCodeZip is used in rank-only mode, where it performs coarse function selection and leaves fine-grained line pruning to

LaMR. Using LongCodeZip in full compression mode before LaMR substantially hurts performance, indicating that coarse retrieval and learned line pruning should be decoupled rather than stacked.

5 Conclusion

We presented LaMR, a role-aware code-context pruner for LLM-powered coding agents. Instead of treating code relevance as a single scalar keep/prune signal, LaMR separates retention into semantic evidence and dependency support, allowing the pruner to preserve self-contained evidence-support units: task-relevant lines together with the structural context needed to interpret them. This formulation is instantiated with rubric-specific CRF heads, query-adaptive gating, AST-derived supervision, and lightweight structural repair. Across SWE-Bench Verified, SWE-QA, LCC, and LongCodeQA, LaMR improves the overall score-token tradeoff over reproduced SWE-Pruner. By effectively filtering irrelevant noise while maintaining structural integrity, the pruned context remains highly competitive with, and occasionally surpasses, uncompressed baselines. Ultimately, LaMR delivers substantial efficiency gains in multi-turn tasks while achieving the strongest task performance among compression methods.

References

- [1] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [2] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. 2024.
- [3] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. 2024.
- [4] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-pruner: Self-adaptive context pruning for coding agents, 2026.
- [5] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- [6] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmllingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [7] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [8] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1):32, 2023.
- [9] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. Longcodezip: Compress long context for code language models. *arXiv preprint arXiv:2510.00446*, 2025.
- [10] Shuai Zheng, Sadeep Jayasumana, Bernardino Romera-Paredes, Vibhav Vineet, Zhizhong Su, Dalong Du, Chang Huang, and Philip HS Torr. Conditional random fields as recurrent neural networks. In *Proceedings of the IEEE international conference on computer vision*, pages 1529–1537, 2015.

- [11] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv preprint arXiv:2203.03850*, 2022.
- [12] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- [13] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *ICLR*, 2024.
- [14] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. Swe-qa: Can language models answer repository-level code questions? *arXiv preprint arXiv:2509.14635*, 2025.
- [15] Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. Longcoder: A long-range pre-trained language model for code completion. In *International Conference on Machine Learning*, pages 12098–12107. PMLR, 2023.
- [16] Stefano Rando, Luca Romani, Alessio Sampieri, Luca Franco, John Yang, Yuta Kyuragi, Fabio Galasso, and Tatsunori Hashimoto. Longcodebench: Evaluating coding llms at 1m context windows. *arXiv preprint arXiv:2505.07897*, 2025.
- [17] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. Llmmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 963–981, 2024.
- [18] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [19] Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36:19327–19352, 2023.
- [20] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, 2023.
- [21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [22] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. Diet code is healthy: Simplifying programs for pre-trained models of code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1073–1084, 2022.
- [23] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, Jiayi Yang, Run Luo, Minzheng Wang, Longze Chen, Junhao Liu, and Min Yang. Hierarchical Context Pruning: Optimizing Real-World Code Completion with Repository-Level Pretrained Code LLMs, 2024.
- [24] Yan Wang, Xiaoning Li, Tien N Nguyen, Shaohua Wang, Chao Ni, and Ling Ding. Natural is the best: Model-agnostic code simplification for pre-trained large language models. *Proceedings of the ACM on Software Engineering*, 1(FSE):586–608, 2024.
- [25] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023.
- [26] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*, pages 933–944, 2018.

- [27] Lipeng Xie, Sen Huang, Zhuo Zhang, Anni Zou, Yunpeng Zhai, Dingchao Ren, Kezun Zhang, Haoyuan Hu, Boyin Liu, Haoran Chen, et al. Auto-rubric: Learning from implicit weights to explicit rubrics for reward modeling. *arXiv preprint arXiv:2510.17314*, 2025.
- [28] Haoxiang Wang, Wei Xiong, Tengyang Xie, Han Zhao, and Tong Zhang. Interpretable preferences via multi-objective reward modeling and mixture-of-experts. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10582–10592, 2024.
- [29] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs Get Lost In Multi-Turn Conversation. 2025.
- [30] Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. Scaling llm multi-turn rl with end-to-end summarization-based context management. *arXiv preprint arXiv:2510.06727*, 2025.
- [31] Cursor. Cursor – the ai code editor, 2025.
- [32] Anthropic. Claude code: Built for developers, 2025.
- [33] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.
- [34] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as llm summarization for agent context management. *arXiv preprint arXiv:2508.21433*, 2025.
- [35] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. Acon: Optimizing context compression for long-horizon llm agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [36] Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. Agentfold: Long-horizon web agents with proactive context management. *arXiv preprint arXiv:2510.24699*, 2025.
- [37] Guangya Wan, Mingyang Ling, Xiaoqi Ren, Rujun Han, Sheng Li, and Zizhao Zhang. Compass: Enhancing agent long-horizon reasoning with evolving context. *arXiv preprint arXiv:2510.08790*, 2025.

A Related Work

Prompt and code-context compression. Token-level pruning methods such as LLMingua [6, 18], Selective-Context [7], and gist-token distillation [19] compress prompts by removing low-saliency tokens or learning compact representations. Retrieval-based approaches select code chunks via embedding similarity or iterative retrieval [20, 11, 21]. Code-specific methods including Long-CodeZip [9], DietCode [22], hierarchical context pruning [23], and SlimCode [24] address structural concerns but are primarily evaluated on single-turn proxy tasks such as code completion [25, 15] or code search [26]. SWE-Pruner [4] introduces learned, query-conditioned line-level pruning with a CRF [10] for structured decisions, demonstrating effectiveness on multi-turn agent benchmarks. Most prior compression methods ultimately make a single keep/drop decision per token, line, or chunk, without explicitly separating direct semantic evidence from dependency support. Recent work on rubric-based reward modeling decomposes evaluation into interpretable dimensions for preference learning [27, 28]. Inspired by this perspective, LaMR decomposes code relevance into multiple latent rubric dimensions, each with its own CRF transition dynamics, enabling the model to retain lines that are structurally necessary even when semantically inconspicuous.

Agent context management. Coding agents [1, 2, 3] face severe context-window pressure in multi-turn workflows, with documented performance degradation on long inputs [5, 29]. Recent trajectory management methods employ LLM summarization [30, 31, 32], context folding [33], observation masking [34], or proactive compression policies [35, 36, 37] to manage prior interaction histories. These approaches are orthogonal to ours: they compress the agent’s *past* trajectories, whereas LaMR compresses the environment’s *current* observations (file content) at the agent–environment boundary, and can be combined with history managers seamlessly.

B Limitations and Broader Impact

Limitations. LaMR is designed for code-observation pruning at the agent–environment boundary, where the agent receives repository files, retrieved snippets, and tool outputs. This scope is complementary to trajectory-level memory or history-compression methods, which manage the agent’s previous tool calls, observations, and reasoning traces. In practice, the two directions can be combined: LaMR reduces the size of incoming code observations, while history-management methods can summarize or organize past interaction context.

Our experiments follow the Python-centered setting of the evaluated coding-agent benchmarks. The semantic/dependency formulation itself is not Python-specific, but applying it to additional languages requires the corresponding parser and dependency extractor. This is a natural engineering extension of the same framework, since the core model only requires rubric labels and does not depend on Python syntax at inference time except for the lightweight repair step.

LaMR uses semantic relevance and dependency support as the deployed learned rubrics. The labeling pipeline can construct additional dimensions such as context, and we study these variants in the ablations. In our experiments, semantic+dependency gives the most reliable tradeoff, while context is better handled through retrieval geometry, local windows, and rendering choices. Future work could explore task-adaptive rubric selection for settings where different interfaces require different forms of surrounding context.

Finally, LaMR is intended as an efficiency layer for coding agents. It reduces unnecessary code context before the downstream model reasons over the task, while preserving fallback behavior: the agent can still request additional files or issue new search commands when more information is needed. This is why we evaluate not only compression, but also total trajectory tokens and interaction rounds.

Broader Impact. LaMR reduces the amount of irrelevant repository context sent to large backbone models, which can lower latency, API cost, and unnecessary long-context computation. This may make repository-level coding agents more accessible to users with limited compute or budget. It may also reduce unnecessary exposure of repository code to external model endpoints by filtering irrelevant observations before inference.

LaMR should not be used as a security filter, a replacement for testing, or a substitute for human review in high-stakes software settings. Its intended use is efficiency-oriented observation compression: helping agents read less irrelevant code while preserving semantic evidence and dependency support needed for reasoning.

C Viterbi Decoding in the CRF Layer

At inference time, the final CRF layer recovers the globally optimal label sequence $\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} S(\mathbf{x}, \mathbf{y})$ via the Viterbi algorithm, a dynamic programming procedure that runs in $O(n \cdot |\mathcal{Y}|^2)$ time (linear in sequence length for our binary label space $|\mathcal{Y}| = 2$).

Forward pass. Let $\mathbf{e}_t^{\text{fused}} \in \mathbb{R}^2$ denote the gated emission at position t , $\mathbf{T}^f \in \mathbb{R}^{2 \times 2}$ the transition matrix, $\boldsymbol{\pi} \in \mathbb{R}^2$ the start potentials, and $\boldsymbol{\omega} \in \mathbb{R}^2$ the end potentials. We maintain a score vector $\mathbf{v}_t \in \mathbb{R}^2$ storing the maximum path score ending in each label at position t , along with backpointers $\mathbf{b}_t \in \{0, 1\}^2$ recording which previous label achieved that maximum:

$$\mathbf{v}_1 = \boldsymbol{\pi} + \mathbf{e}_1^{\text{fused}}, \quad (8)$$

$$v_{t,j} = \max_{i \in \{0,1\}} (v_{t-1,i} + T_{j,i}^f) + e_{t,j}^{\text{fused}}, \quad t = 2, \dots, n, \quad (9)$$

$$b_{t,j} = \arg \max_{i \in \{0,1\}} (v_{t-1,i} + T_{j,i}^f). \quad (10)$$

Termination and backtracking. After processing all n positions, the optimal final label is $\hat{y}_n = \arg \max_j (v_{n,j} + \omega_j)$. The full sequence is recovered by following backpointers in reverse: $\hat{y}_{t-1} = b_{t,\hat{y}_t}$ for $t = n, \dots, 2$. The output $\hat{\mathbf{y}} \in \{0, 1\}^n$ is a binary keep/prune decision for each token, where 0 denotes prune and 1 denotes keep.

Unlike independent per-token classification, Viterbi decoding considers the transition preferences between adjacent labels, producing coherent pruning patterns that respect code block boundaries. For example, a high $T_{1,1}^f$ (keep $\rightarrow$ keep) score encourages contiguous retained regions, avoiding isolated single-token keep decisions that would fragment the output.

D Experimental Details

D.1 Benchmark Descriptions

SWE-Bench Verified [13] is a curated subset of 500 real-world GitHub issues drawn from popular Python repositories. Each instance requires an agent to navigate the repository, understand the issue, and generate a patch that resolves the problem. We integrate LaMR into the Mini SWE Agent [1] framework, which provides a ReAct-style agent with terminal access for file reading, code search, and patch submission. Success is measured by whether the generated patch passes the repository’s test suite.

SWE-QA [14] evaluates repository-level question answering across three Python repositories. Given a natural-language question about the codebase, the agent must locate and synthesize information from multiple files. We use the OpenHands [2] framework for this benchmark. Answers are scored by an LLM-as-a-Judge on a scale of 0–10.

Long Code Completion (LCC) [15] contains 500 Python code completion examples with contexts exceeding 5K tokens. The model must predict the next code segment given a long preceding context. Performance is measured by Edit Similarity (ES) and Exact Match (EM).

Long Code QA (LongCodeQA) [16] tests question answering over code contexts up to 1M tokens. Given a question about a large codebase, the model must select or generate the correct answer. Performance is measured by Accuracy.

Both single-turn benchmarks are evaluated under $4\times$ and $8\times$ compression constraints, where all methods receive the same target compression ratio.

D.2 Baseline Descriptions

We compare against the following methods, all configured under identical compression constraints:

- **Full Context**: The agent receives the complete, uncompressed context. Serves as the upper bound for task performance.
- **No Context**: The agent receives no file content. Serves as the lower bound.
- **LLMLingua-2** [17]: Token-level pruning via a learned perplexity model. Compresses prompts by removing tokens with the lowest perplexity contribution.
- **Selective-Context** [7]: Token-level pruning based on self-information. Removes tokens with low information content as estimated by a language model.
- **RAG** [11]: Retrieval-augmented generation using UniXCoder embeddings. Retrieves the most relevant code chunks via embedding similarity to the query.
- **LongCodeZip** [9]: Program-structure-aware compression that leverages AST information to identify and remove redundant code elements.
- **LLM Summarize** (multi-turn only): Generates abstractive summaries of file contents using the agent’s backbone LLM. Incurs additional inference cost.
- **SWE-Pruner** [4]: Single-objective CRF-based pruner with the Qwen3-Reranker-0.6B backbone. This is the direct baseline that LaMR extends with latent multi-rubric decomposition and expert gating.

We do not compare with agent history compression methods such as AgentFold [36] and ACON [35], as they address the complementary problem of compressing prior interaction trajectories rather than environment observations (file content).

D.3 Metric Definitions

Task performance metrics:

- **Resolve Rate** (SWE-Bench Verified): Fraction of issues for which the generated patch passes all repository test cases.
- **LLM-as-a-Judge Score** (SWE-QA): Average score (0–10) assigned by an LLM judge evaluating the correctness and completeness of the agent’s answer.
- **Edit Similarity (ES)** (LCC): Character-level edit similarity between the predicted completion and the ground-truth continuation, computed as $1 - \text{edit_distance}(y, \hat{y}) / \max(|y|, |\hat{y}|)$.
- **Exact Match (EM)** (LCC): Fraction of examples where the predicted completion exactly matches the ground truth.
- **Accuracy** (LongCodeQA): Fraction of questions answered correctly.

Compression efficiency metrics:

- **Compression Ratio** (ρ): $\rho = |C_{\text{original}}| / |C_{\text{compressed}}|$, where $|\cdot|$ denotes token count. Higher values indicate more aggressive compression.
- **Token Consumption**: Total number of input tokens consumed across all interactions (multi-turn) or per instance (single-turn).
- **Rounds** (multi-turn only): Number of agent interaction rounds required to complete the task. Fewer rounds indicate more decisive reasoning.

D.4 Hyperparameters

Table 5 lists all hyperparameters used for training LaMR.

D.5 Full Training Objective

The complete training objective extends the core loss (eq. (7)) with two auxiliary terms:

$$\mathcal{L}_{\text{full}} = (1 - \lambda_s) \left[(1 - \lambda_r) \mathcal{L}_{\text{main}} + \lambda_r \mathcal{L}_{\text{rubric}} \right] + \lambda_s \mathcal{L}_{\text{score}} + \lambda_g \mathcal{L}_{\text{gate}} \quad (11)$$

Table 5: Training hyperparameters for LaMR.

Hyperparameter	Value
Backbone	Qwen3-Reranker-0.6B [12]
Fine-tuned layers	Top 2 transformer layers
Fusion layers (N_{fuse})	1
Number of objectives (K)	2
Objective names	semantic, dependency
Rubric weights (w_k)	1.0, 0.7
Gating type	Softmax
Use final CRF	Yes
λ_r (rubric weight)	0.6
λ_g (gate entropy weight)	0.002
Learning rate	1×10^{-4}
Batch size (per GPU)	16
Dropout	0.4
Loss function	CRF-NLL + MSE score loss + gate regularization
GPUs	8
Optimizer	AdamW

Document-level relevance loss ($\mathcal{L}_{\text{score}}$, $\lambda_s = 0.05$). Following SWE-Pruner, we add a document-level scoring head that predicts overall relevance from the last token’s hidden state projected onto the vocabulary embeddings. The loss is the MSE between the predicted probability (softmax over the yes/no logits) and the ground-truth relevance score. This auxiliary task encourages the backbone to maintain a global view of document relevance alongside the token-level pruning decisions.

Gate-balance regularization ($\mathcal{L}_{\text{gate}}$, $\lambda_g = 0.002$). To prevent the gating network from collapsing to a single dominant rubric, we add a gate-balance loss $\mathcal{L}_{\text{gate}}$ as defined in Section 3. This term encourages high entropy in the gate distribution averaged over valid positions, promoting balanced use of the semantic and dependency rubrics during training.

E Proofs

Proof of proposition 1. If $e \in S$ and $d \notin S$, adding d removes the missing-support penalty M and pays one line cost λ :

$$\Delta(d \mid e \in S) = M - \lambda > 0.$$

If $e \notin S$, adding d provides no evidence benefit and only pays cost:

$$\Delta(d \mid e \notin S) = -\lambda < 0.$$

The value of a support line thus depends on whether the evidence it supports is retained. A single unconditional scalar score cannot represent both cases. $\square$

Proof of proposition 2. Assume for contradiction that an optimal S^* is not dependency-closed. Then there exists $(i, j) \in E$ with $i \in S^*$ and $j \notin S^*$, so S^* pays at least one penalty M . Consider $S' = S^* \cup \{j\}$. Adding j removes at least one penalty of size M . The cost-adjusted contribution is bounded by $(r_j - \lambda c_j)_+$. By assumption $M > (r_j - \lambda c_j)_+$, so $U(S') > U(S^*)$, contradicting optimality. $\square$

F Full Ablation Results

This appendix expands the ablation summary in Section 4.3. We report full results for four analyses: (i) validation objective/architecture ablation, (ii) downstream LongCodeQA ablation, (iii) SWE-QA Conan ablation, and (iv) robustness to upstream context constructors. All variants use the same Qwen3-Reranker backbone and training pipeline unless otherwise specified. For validation results, the FFN and CRF losses are on different numerical scales, so we compare primarily accuracy, precision, recall, and F1.

Table 6: Claim-evidence map for the ablation study. The table summarizes which experiments support each design choice; full numerical results are reported in the following appendix tables.

Claim	Supporting evidence
Semantic relevance alone is insufficient.	Semantic-only underperforms the full model on validation F1 (0.777 vs. 0.796), LongCodeQA, and Conan. This indicates that direct relevance alone misses support lines needed for useful compressed code.
Dependency is the best learned complement to semantic relevance among tested objectives.	Semantic+dependency with AST outperforms semantic+context, context+dependency, and semantic+dependency+context on validation F1 and downstream LongCodeQA.
Context is useful but unstable as a universal learned objective.	Context variants improve recall or compression in some settings, but degrade under $8\times$ LongCodeQA and underperform the full model on Conan. This supports handling context through retrieval/rendering rather than as an always-on objective.
CRF structure matters.	In the three-objective setting, CRF improves over FFN in recall and F1 (.784 vs. .755), showing that structured sequence modeling is important for pruning.
AST-aware structure matters.	Semantic+dependency w/ AST beats w/o AST on validation F1 (.796 vs. .780), LongCodeQA $8\times$ accuracy (60.00 vs. 55.56), and Conan score/round tradeoff (8.48/18.8 vs. 8.37/21.3).
LaMR improves the score-token tradeoff.	Main results show strong single-turn gains on LCC and LongCodeQA, and lower token/round cost than reproduced SWE-Pruner in multi-turn agent settings.
Coarse retrieval and learned line pruning should be decoupled.	LongCodeZip rank-only + LaMR achieves the strongest LCC $8\times$ result, while LongCodeZip full compression before LaMR substantially hurts quality.

To make the ablation logic explicit, Table 6 maps each methodological claim to the experiment that supports it. This table is intended as a guide to the detailed ablation results below, not as an additional evaluation metric.

F.1 Objective and Architecture Ablation

Table 7: Objective and architecture ablation on validation data. The deployed semantic+dependency model with AST-aware repair achieves the best F1 and accuracy. Context improves recall in some configurations but does not improve the final tradeoff. The FFN three-objective model is high precision but low recall, while CRF structure recovers recall and improves F1.

Objective Configuration	Head	Epoch	C-Loss ↓	Acc. ↑	Prec. ↑	Rec. ↑	F1 ↑
Semantic only	CRF	3	0.3399	0.8052	0.7682	0.7861	0.7770
Semantic + Context	CRF	3	0.3715	0.8144	0.7789	0.7963	0.7875
Dependency + Context	CRF	3	0.3931	0.8083	0.7694	0.8033	0.7860
Semantic + Dependency + Context	FFN	3	0.0532	0.8029	0.8297	0.6921	0.7547
Semantic + Dependency + Context	CRF	3	0.3720	0.8115	0.7863	0.7826	0.7844
Semantic + Dependency w/o AST	CRF	3	0.3669	0.8252	0.7929	0.7683	0.7804
Semantic + Dependency w/ AST	CRF	3	0.3620	0.8253	0.8154	0.7772	0.7958

Semantic-only pruning achieves reasonable precision but remains below LaMR in F1, indicating that direct relevance alone does not recover all useful support lines. Context-based variants improve recall in some settings but do not outperform semantic+dependency. The three-objective setting also does not improve over the two-objective deployed model, supporting the choice of semantic and dependency as the learned rubrics. The FFN variant is highly precise but recall-limited, whereas CRF structure improves recall and overall F1.

Table 8: LongCodeQA downstream ablation under $4\times$ and $8\times$ compression constraints. LaMR achieves the best accuracy and compression in both settings. Context-based objectives are competitive at $4\times$ but unstable under $8\times$ compression, while removing AST structure substantially reduces $8\times$ accuracy.

Pruner Variant	4× Constraint		8× Constraint	
	$1/\tau \uparrow$	Acc. $\uparrow$	$1/\tau \uparrow$	Acc. $\uparrow$
Semantic only	15.92	56.25	17.27	58.55
Semantic + Context	12.13	59.26	16.75	56.48
Context + Dependency	11.73	60.36	15.06	51.82
Semantic + Dependency w/o AST	12.99	59.09	18.00	55.56
Semantic + Dependency + Context	11.87	57.27	15.00	53.21
LaMR	16.48	61.26	18.51	60.00

F.2 LongCodeQA Downstream Ablation

The LongCodeQA ablation shows that the validation trends transfer to a downstream long-code QA task. Context-based objectives are not consistently robust under stronger compression. Removing AST-aware structure especially hurts under the $8\times$ constraint, suggesting that structural support becomes more important as the token budget tightens.

F.3 SWE-QA Conan Ablation

Table 9: SWE-QA Conan ablation with Claude Sonnet 4.5. LaMR achieves the highest answer score and the fewest interaction rounds while reducing token usage by 20.5% relative to the unpruned baseline. Context-heavy variants compress more aggressively but reduce answer quality, while removing AST structure lowers score and increases rounds compared with LaMR.

Method	Avg Score		Rounds	Tokens (K)	
	Value	Δ	Value	Value	Δ
<i>Conan, Claude Sonnet 4.5</i>					
Unpruned	8.35	—	23.9	672.9	—
+ Semantic only	8.13	$\downarrow 0.22$	21.6	515.2	$\downarrow 23.4\%$
+ Semantic + Context	8.26	$\downarrow 0.09$	21.3	480.6	$\downarrow 28.6\%$
+ Context + Dependency	8.32	$\downarrow 0.03$	21.3	519.4	$\downarrow 23.5\%$
+ Semantic + Dependency w/o AST	8.37	$\uparrow 0.02$	21.3	494.4	$\downarrow 26.5\%$
+ Semantic + Dependency + Context	8.31	$\downarrow 0.04$	21.6	514.9	$\downarrow 23.5\%$
+ LaMR	8.48	$\uparrow 0.13$	18.8	535.2	$\downarrow 20.5\%$

This ablation highlights the difference between aggressive compression and useful compression. Semantic+context uses fewer tokens than LaMR, but lowers answer quality. Semantic+dependency without AST also uses fewer tokens than LaMR, but requires more rounds and obtains a lower score. LaMR retains slightly more context than the most aggressive variants, but the retained context is more useful to the agent, leading to higher score and fewer rounds.

F.4 Robustness Across Context Constructors

These results support a coarse-to-fine interpretation of LaMR. The upstream constructor should preserve candidate recall, while LaMR performs fine-grained semantic/dependency line pruning. When the upstream constructor also performs line-level compression, useful evidence may be removed before LaMR can score it.

Summary of ablations. Together, the validation, LongCodeQA, and SWE-QA Conan ablations support the same conclusion. Semantic relevance identifies direct evidence, but dependency supervision is needed to preserve structural support. Context is useful but unstable as a universal learned

Table 10: Robustness across upstream context constructors on LCC $8\times$. LaMR is effective with both RAG and LongCodeZip retrieval. The best result is obtained when LongCodeZip is used in rank-only mode, which performs coarse function selection and leaves fine-grained line pruning to LaMR. Using LongCodeZip in full compression mode before LaMR substantially hurts performance, showing that stacking two line compressors is harmful.

Constructor	Pruner	EM $\uparrow$	ES $\uparrow$	$1/\tau$ $\uparrow$
LCZP (rank-only)	LaMR	34.5	59.21	9.17
LCZP (rank-only)	SWE-Pruner	13.0	43.99	10.67
LCZP (full)	LaMR	24.0	52.39	10.62
RAG ($k = 5$)	LaMR	31.0	57.27	7.53
RAG ($k = 5$, wide)	LaMR	31.0	58.26	6.52

objective; it is better handled by retrieval, local windows, and rendering. CRF-structured prediction improves recall over independent FFN classification, and AST-aware repair improves both validation and downstream performance, especially under tighter compression.